Roteiro

- Tipos Abstratos de Dados
  - Tipo de Dados
  - Tipos Abstratos de Dados
  - Tipos Abstratos de Dados Definidos pelo Usuário
- TAD
  - Lista
  - Pilha
  - Fila

...

Tipos Abstratos de Dados

Tipo de Dados

Tipos Abstratos de Dados

Tipos Abstratos de Dados Definidos pelo Usuário

Tipos de Dados

- Quando se fala sobre um ***tipo primitivo*** , na verdade está se referindo a duas coisas:
  - Um ***item de dados*** com certas características
  - ***Operações*** permitidas nessas dados
- Tipos simples de dados são grupos de ***valores indivisíveis*** (como os tipos básicos ***integer***, ***boolean***, ***char*** e ***real*** do Pascal).
  - Exemplo: uma variável do tipo *boolean* pode assumir o valor verdadeiro ou o valor falso, e nenhum outro valor.
- Os Tipos Estruturados
  - Em geral definem uma coleção de valores simples, ou um agregado de valores de tipos diferentes.

Tipos Abstratos de Dados

- Um TAD é um ***encapsulamento*** que inclui somente a representação de dados de um tipo específico de dado e os subprogramas que fornecem as ***operações*** para esse tipo
  - Detalhes desnecessários do tipo podem ser ocultados das unidades fora do ***encapsulamento*** que o usam
  - As unidades de programas que usam dados abstratos podem ***declarar variáveis desse tipo***

Tipos Abstratos de Dados

- Motivação
  - ***Diminuir a complexidade***, um meio de tornar programas grandes e/ou complicados mais manejáveis
  - Método de organizar um programa em unidades lógicas que podem ser ***compiladas separadamente***
  - Permite que modificações nas representações ou operações do tipo sejam feitas em ***única área do programa***
  - A interface para a abstração representa alguns, mas não todos, os seus ***atributos e operações***
  - ***Confiabilidade***
  - Modificações podem ocorrer a qualquer hora sem exigir mudanças no cliente.
    - Exemplo: alteração do TAD de representação por ***adjacência para encadeada***.

Tipos Abstratos de Dados
Definidos pelo Usuário

- Um TAD é um tipo de dado que satisfaz as duas condições seguintes:
  - A representação ou a definição do tipo e as operações sobre objetos do tipo estão contidas em ***um única unidade sintática***.
  - A representação de objetos do tipo ***não é visível*** pelas unidades de programa que usam o tipo, de modo que as únicas operações diretas possíveis sobre esse objetos são aquelas ***oferecidas na definição do tipo***.

...

TADs

Listas Lineares

TAD – Listas Lineares
Definição

- Seqüência de zero ou mais itens $x_1, x_2, \ldots, x_n$, na qual x_i é de um determinado tipo e n representa o tamanho da ***lista linear***.

- Sua principal propriedade estrutural envolve as ***posições relativas*** dos itens em uma dimensão.
  - Assumindo $n \geq 1$, x_1 é o primeiro item da lista e x_n é o último item da lista.
  - x_i precede x_i+1 para $i = 1,2,..,n-1$
  - x_i sucede x_i-1 para $i = 2,3,\ldots,n$
  - o elemento x_i é dito estar na i-ésima posição da lista.

TAD – Listas Lineares
Aplicações

- Lista de Pagamentos
  - Prestação do carro
  - Cartão de crédito
  - Conta de luz
  - Condomínio
  - TV a cabo
  - Prestação do crediário

TAD – Listas Lineares
Operações

- O conjunto de operações a ser definido depende de cada aplicação.
- Um conjunto de operações a ser disponibilizada ao cliente neste exemplo serão as seguintes:

```
Cria            (Lista);
InsereInicio    (Lista,Item);
InserePosicao   (Lista,p,Item);
InsereFinal     (Lista,Item);
RemoveInicio    (Lista);
RemoveFinal     (Lista);
Pesquisa_Remove (Lista,Item);
Imprime         (Lista);
```

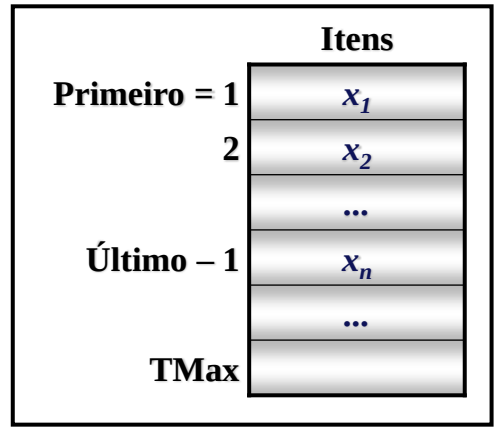

TAD – Listas Lineares

Implementação por meio de Arranjos

TAD – Listas Lineares
Implementação por meio de Arranjos

- Os itens da lista são armazenados em ***posições contíguas de memória***.
- A lista pode ser percorrida em ***qualquer direção***.
- A inserção de um novo item pode ser realizada após o último item com ***custo constante***.
- A inserção de um novo item no meio da lista requer um ***deslocamento de todos*** os itens localizados após o ponto de inserção.
- Retirar um item do início da lista requer um ***deslocamento de itens*** para preencher o espaço deixado vazio.

TAD – Listas Lineares
Implementação por meio de Arranjos



TAD – Listas Lineares
Implementação por meio de Arranjos

- Os itens são armazenados em um ***array*** de tamanho suficiente para armazenar a lista.
- O campo ***Último*** aponta para a posição seguinte a do último elemento da lista.
- O i-ésimo item da lista está armazenado na i-ésima posição do ***array***, $1 \leq i \leq Último$.
- A constante ***TMax*** define o tamanho máximo permitido para a lista.

TAD – Listas Lineares - Implementação por meio de Arranjos

- **Estrutura em C**

```
const int Tmax = 10;

typedef int Apontador;

typedef struct  {
    int Chave;
    /*outros componentes*/
} TInfo;

typedef struct {
    TInfo Item[Tmax];
    Apontador Primeiro;
    Apontador Ultimo;
} TLista;
```

TAD – Listas Lineares - Implementação por meio de Arranjos

- **Subrotinas** *Interface*

```
Cria              (Lista);
InsereInicio      (Lista,Item);
InserePosicao     (Lista,p,Item);
InsereFinal       (Lista,Item);
RemoveInicio      (Lista);
RemoveFinal       (Lista);
Pesquisa_Remove   (Lista,Item);
Imprime           (Lista);
```

deslocamento

TAD – Listas Lineares - Implementação por meio de Arranjos

```
void InsereInicio (TLista& Lista, TInfo Item)
{

  if (Cheia(Lista))
     cout << "Erro: A lista está cheia" << endl;
  else
  {

    for (int aux=Lista.Ultimo; aux>=Lista.Primeiro+1; aux--)
        Lista.Item[aux] = Lista.Item[aux-1];

    Lista.Item[Lista.Primeiro] = Item;
    Lista.Ultimo++;

  }
}
```

TAD – Listas Lineares - Implementação por meio de Arranjos

Tmax = 10

		0	1	2	3	4	5	6	7	8	9
Lista	item	15	2	40	5	14	18	31	25		

primeiro **0** *ultimo* **8**

```
InsereInicio (Lista,Item); //item.chave := 0
```

		0	1	2	3	4	5	6	7	8	9
Lista	item	0	15	2	40	5	14	18	31	25	

primeiro **0** *ultimo* **9**

TAD – Listas Lineares
Implementação por meio de Arranjos

- Vantagem
  - ***economia de memória*** (os apontadores são implícitos nesta estrutura).
- Desvantagens:
  - custo para inserir ou retirar itens da lista, que pode causar um ***deslocamento*** de todos os itens, no pior caso;
  - em aplicações em que não existe previsão sobre o crescimento da lista, a utilização de arranjos em linguagens como o Pascal pode ser problemática porque neste caso o ***tamanho máximo da lista*** tem de ser definido em tempo de compilação.

TAD – Listas Lineares

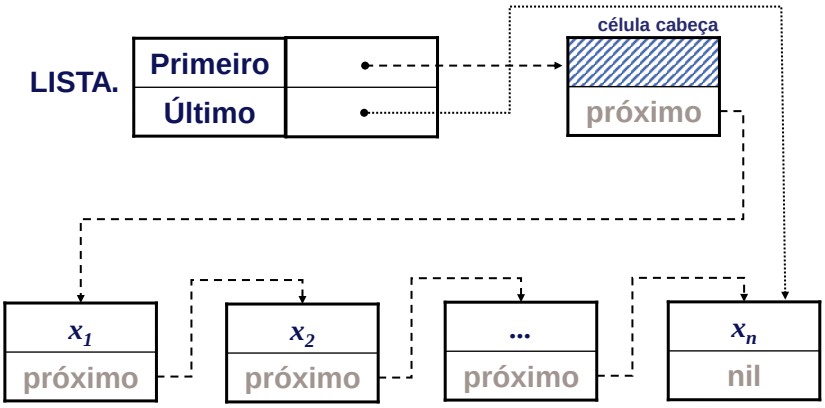
Implementação por meio de Apontadores

TAD – Listas Lineares
Implementação por meio de Apontadores

- Cada item é ***encadeado*** com o seguinte mediante uma variável do tipo Apontador.
- Permite utilizar ***posições não contíguas*** de memória.
- É possível inserir e retirar elementos ***sem*** necessidade de ***deslocar*** os itens seguintes da lista.

TAD – Listas Lineares
Implementação por meio de Apontadores

- Há uma ***célula cabeça*** para simplificar as operações sobre a lista.
- A parte ***info*** desse nó poderia fica sem uso, conforme abaixo.
- Porém pode ser usado para manter ***informações globais*** sobre a lista inteira.



TAD Listas Lineares - Implementação por meio de Apontadores

- **Estrutura em C**

```
typedef struct NodoLista *Apontador;

typedef struct  {
    int Chave;
    /*outros componentes*/
} TInfo;

struct NodoLista{
    TInfo Item;
    struct NodoLista *Proximo;
};

typedef struct {
    struct NodoLista *Primeiro;
    struct NodoLista *Ultimo;
} TLista;
```

TAD Listas Lineares - Implementação por meio de Apontadores

- **Subrotinas** *Interface*

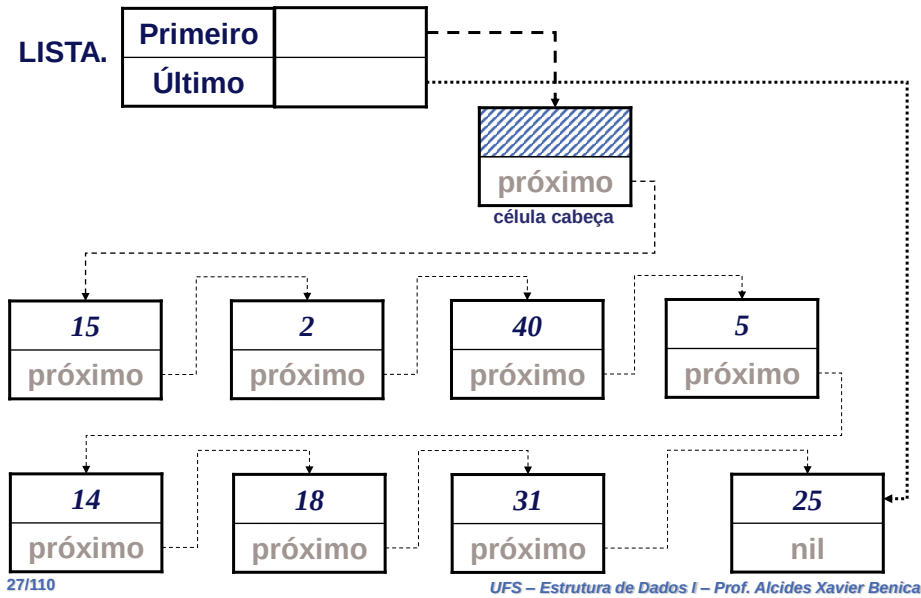
```
Cria              (Lista);
InsereInicio      (Lista,Item);
InserePosicao     (Lista,p,Item);
InsereFinal       (Lista,Item);
RemoveInicio      (Lista);
RemoveFinal       (Lista);
Pesquisa_Remove   (Lista,Item);
Imprime           (Lista);
```

nenhum deslocamento

TAD Listas Lineares - Implementação por meio de Apontadores

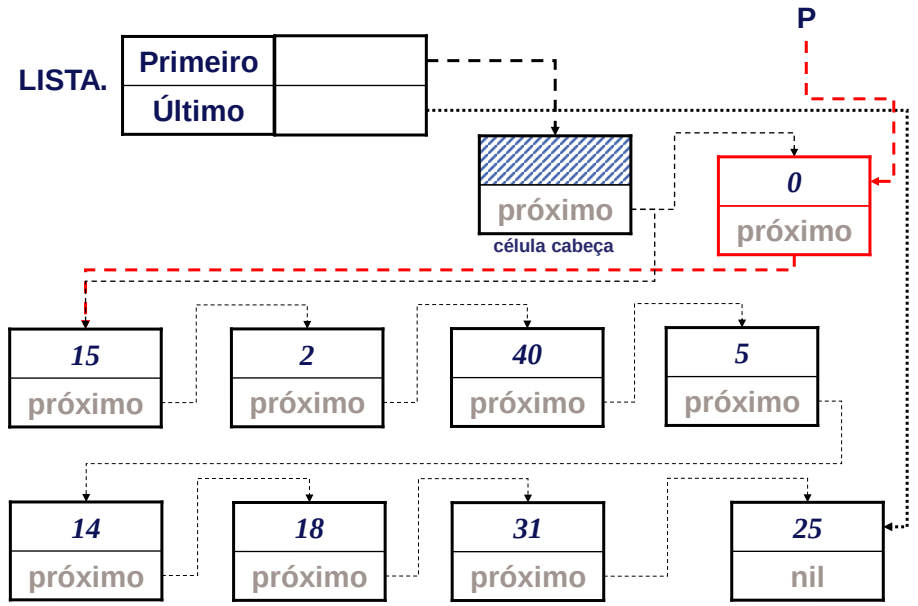
```
void InsereInicio (TLista& Lista, TInfo Item) {

  Apontador p = (Apontador) malloc(sizeof(struct NodoLista));
  p->Item = Item;
  p->Proximo = Lista.Primeiro->Proximo;
  Lista.Primeiro->Proximo = p;
  if (p->Proximo == NULL)
      Lista.Ultimo = p;

}
```

TAD Listas Lineares - Implementação por meio de Apontadores



TAD Listas Lineares - Implementação por meio de Apontadores

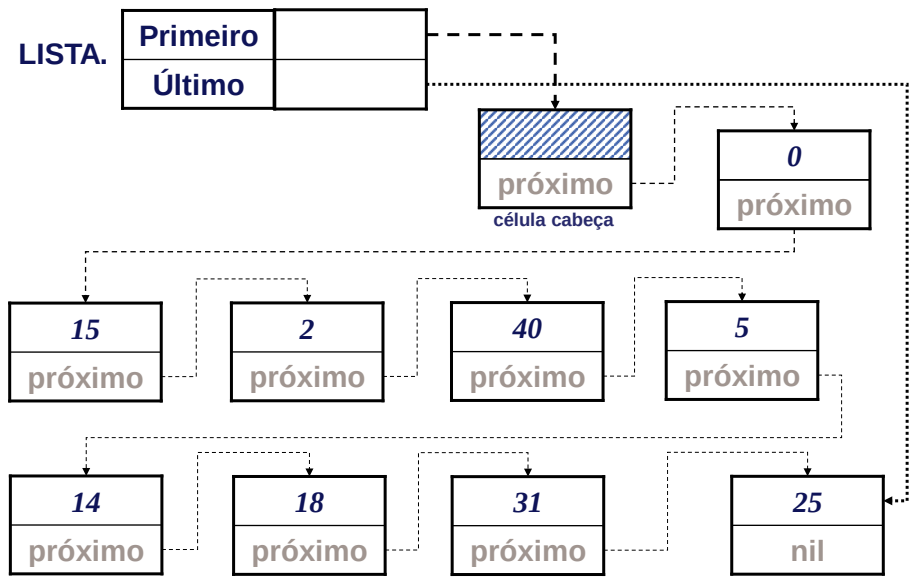
```
InsereInicio (Lista,Item); //item.chave := 0
```



TAD Listas Lineares - Implementação por meio de Apontadores

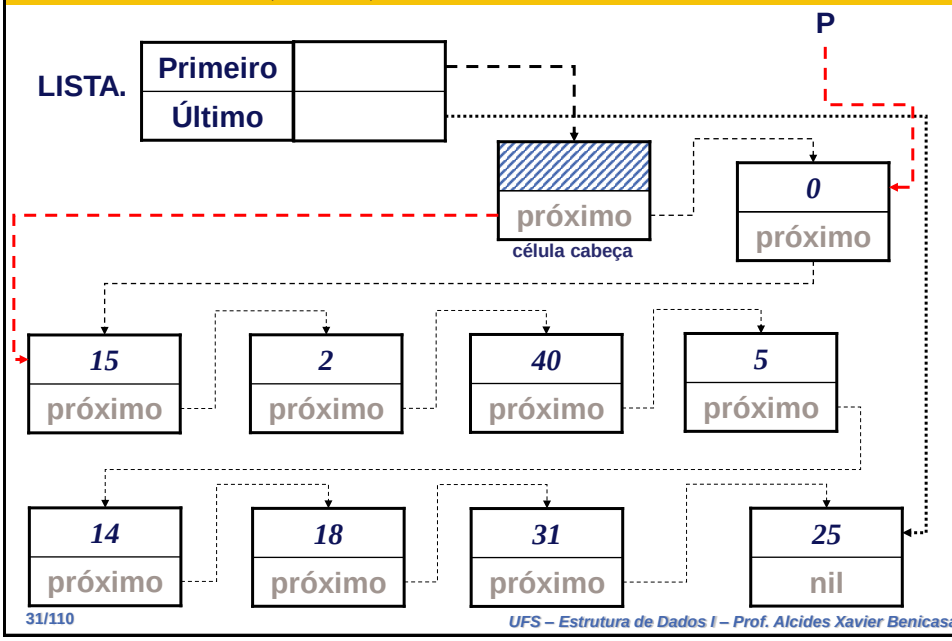
```
void RemoveInicio (TLista& Lista) {
 if (Vazia(Lista))
    cout << "Erro : A lista está vazia.\n" << endl;
 else {
   Apontador p = Lista.Primeiro->Proximo;
   cout << "Elemento Removido=" << p->Item.Chave << endl;
   Lista.Primeiro->Proximo = p->Proximo;
   if (Lista.Primeiro->Proximo == NULL)
      Lista.Ultimo = Lista.Primeiro;
      free(p);
 }
}
```


TAD Listas Lineares - Implementação por meio de Apontadores



TAD Listas Lineares - Implementação por meio de Apontadores

`RemoveInicio (Lista);`



TAD – Listas Lineares
Implementação por meio de Apontadores

- Vantagens:
  - Permite inserir ou retirar itens do meio da lista a um ***custo constante*** (importante quando a lista tem de ser mantida em ordem).
  - Bom para aplicações em que não existe previsão sobre o ***crescimento da lista*** (o tamanho máximo da lista não precisa ser definido *a priori*).
- Desvantagem: utilização de memória extra para ***armazenar os apontadores***.

TAD – Listas Lineares Duplamente Encadeadas

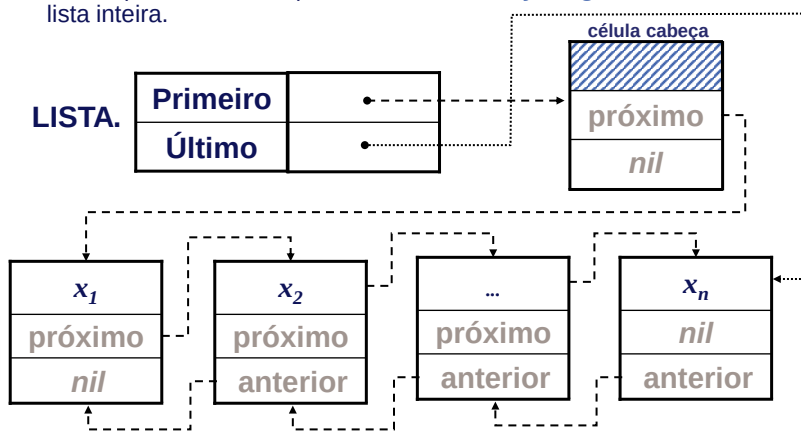
Implementação por meio de Apontadores

TAD – Listas Lineares Duplamente Encadeadas

- A Lista Encadeada e possue a desvantagem de somente podermos ***caminhar em uma direção***.
- A ***Lista Duplamente Encadeada*** é uma estrutura de lista que permite deslocamento em ambos os sentidos:
  - Útil para representar conjuntos de eventos ou objetos a serem percorridos em dois sentidos.
  - Ex.: Itinerários de ônibus, trem ou avião.
  - Útil também quando realizamos uma busca aproximada e nos movemos para e frente e trás.

TAD – Listas Lineares
Duplamente Encadeadas

- Há uma ***célula cabeça*** para simplificar as operações sobre a lista.
- A parte ***info*** desse nó poderia fica sem uso, conforme abaixo.
- Porém pode ser usado para manter ***informações globais*** sobre a lista inteira.



TAD Listas Lineares – Duplamente Encadeada

- **Estrutura em C**

```
typedef struct NodoLista *Apontador;

typedef struct  {
    int Chave;
    /*outros componentes*/
} TInfo;

struct NodoLista{
    TInfo Item;
    struct NodoLista *Anterior;
    struct NodoLista *Proximo;
};

typedef struct {
    struct NodoLista *Primeiro;
    struct NodoLista *Ultimo;
} TLista;
```

TAD Listas Lineares – Duplamente Encadeada

- **Subrotinas** *Interface*

```
Cria              (Lista);
InsereInicio      (Lista,Item);
InserePosicao     (Lista,p,Item);
InsereFinal       (Lista,Item);
RemoveInicio      (Lista);
RemoveFinal       (Lista);
Pesquisa_Remove   (Lista,Item);
Imprime           (Lista);
```

nenhum deslocamento